

lab_emnist_partial

November 3, 2025

1 Lab: SVMs on Extended MNIST

In the [MNIST demo](#), we saw how SVMs can be used for the classic MNIST problem of digit recognition. In this lab, we are going to extend the MNIST dataset by adding non-digit letters and see if the classifier can distinguish the digits from the non-digits. All non-digits will be lumped as a single 11-th class. This is a highly simplified version of a full character classification, but will illustrate some concepts.

In addition to the concepts in the demo, you will learn: * Select the SVM parameters (C and gamma) via cross-validation. * Use the `GridSearchCV` method to search for parameters with cross-validation.

As usual, we download the standard packages

```
[64]: import numpy as np
import matplotlib.pyplot as plt
from sklearn import linear_model, preprocessing
```

1.1 Downloading the EMNIST data

The EMNIST data can be downloaded from `pytorch` as follows. The initial download is 562 MB, so it may take a little while. On Google colab, it will be in less than a minute.

Note: In an earlier version of the lab, you directly downloaded the data from the [EMNIST](#), but the dataset appears to have been removed.

```
[65]: import torch
import torchvision.datasets as datasets
import torchvision.transforms as transforms

# Define the transform to convert the images to tensors
transform = transforms.Compose([transforms.ToTensor()])

# Download the EMNIST dataset
emnist_dataset = datasets.EMNIST(root='./data', split='balanced', train=True,
    ↪transform=transform, download=True)
```

You can then convert the data to numpy arrays as follows.

* `Xtr` is `(ntr,28,28)` the training data array representing 28x28 images with `ntr` samples. *

`ytr[i]` is the integer label from 0 to `nclasses-1`. When `ytr[i] = k` it corresponds to the string digit `n_classes[k]`.

```
[66]: Xtr = emnist_dataset.train_data.numpy()
      ytr = emnist_dataset.targets.numpy()
      Xts = emnist_dataset.train_data.numpy()
      yts = emnist_dataset.targets.numpy()
      classes = emnist_dataset.classes
```

```
[67]: # TODO: Print the number of training and test samples
      ntr = Xtr.shape[0]
      nts = Xts.shape[0]

      print(ntr, nts)
```

112800 112800

Print the classes. You should see there are classes for: * digits 0 to 9 * capital letters A to Z * some lowercase letters: `a`,`b,d,e,f,g,h,n,q,r,t``

Not all lowercase letters are included since they may be too easy to confuse with the uppercase.

```
[68]: # TODO: Print the classes and the total number of classes
      print(classes)
      nclasses = len(classes)
      print(nclasses)
```

['0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z', 'a', 'b', 'd', 'e', 'f', 'g', 'h', 'n', 'q', 'r', 't']
47

1.2 Displaying the Characters

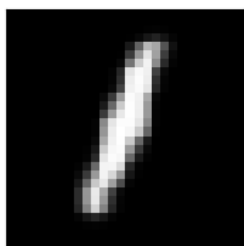
We will use the function from the demo to plot the characters. Note that the `label` argument takes either `None` or a string, not a decimal.

```
[69]: def plt_digit(x,label=None):
      nrow = 28
      ncol = 28
      xsq = x.reshape((nrow,ncol))
      plt.imshow(xsq.T, cmap='Greys_r')
      plt.xticks([])
      plt.yticks([])
      if label:
          plt.title(label)
```

Plot 8 random samples from the training data. You can use the `plt_digit` function above with `subplot` to create a nice display. You may want to size your plot with the `plt.figure(figsize=(10,20))` command.

```
[70]: # TODO: Plot 8 random samples from the training data of the digits  
# Select random digits  
digits = np.random.randint(0,10,8)  
plt.figure(figsize=(10,20))  
for i in range(8):  
    plt.subplot(2,4,i+1)  
    plt_digit(Xtr[digits[i]], classes[ytr[digits[i]]])  
plt.show()
```

1



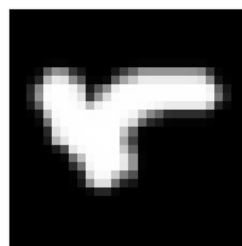
n



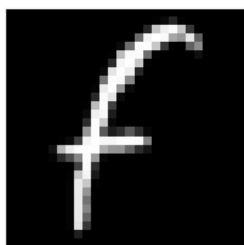
n



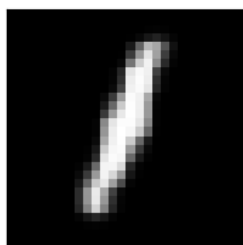
r



F



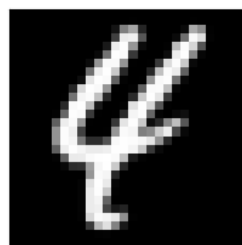
1



n



4



1.3 Creating a Non-Digit Class

Before we begin, we first need to remove all the letters corresponding to I, L and O since they are too similar to the other characters. Loop over the elements in `remove_list` and delete the rows of `Xtr`, `Xts`, `ytr`, and `yts` corresponding to these characters.

```
[87]: # Reload data first to fix the mismatch
Xtr = emnist_dataset.data.numpy()
ytr = emnist_dataset.targets.numpy()
Xts = emnist_dataset.data.numpy()
yts = emnist_dataset.targets.numpy()

# Now remove I, L, O
remove_list = ['I', 'L', 'O']
remove_indices = [list(classes).index(r) for r in remove_list]

print(f"Removing labels: {remove_indices} (corresponding to {remove_list})")
print(f"Original shapes: Xtr={Xtr.shape}, ytr={ytr.shape}, Xts={Xts.shape},  

      yts={yts.shape}")

# Create mask and apply
mask_tr = ~np.isin(ytr, remove_indices)
mask_ts = ~np.isin(yts, remove_indices)

Xtr = Xtr[mask_tr]
ytr = ytr[mask_tr]
Xts = Xts[mask_ts]
yts = yts[mask_ts]

print(f"After removal: Xtr={Xtr.shape}, ytr={ytr.shape}, Xts={Xts.shape},  

      yts={yts.shape}")
```

```
Removing labels: [18, 21, 24] (corresponding to ['I', 'L', 'O'])
Original shapes: Xtr=(112800, 28, 28), ytr=(112800,), Xts=(112800, 28, 28),
yts=(112800,)
After removal: Xtr=(105600, 28, 28), ytr=(105600,), Xts=(105600, 28, 28),
yts=(105600,)
```

SVM classifiers are VERY SLOW to train. The training is particularly slow when there are a large number of classes, since the one classifier must be trained for each pair of labels. To make the problem easier, we are going to lump all of the letters in one class and add that class to the digits.

We can do this by creating a label for the non-digit class `non_digit_class = 10`. Then whenever `ytr[i] >= non_digit_class` we set `ytr[i] = non_digit_class`. We do the same with `yts`.

```
[88]: non_digit_class = 10

# Relabel all non-digit classes (>= 10) to class 10
ytr[ytr >= non_digit_class] = non_digit_class
```

```

yts[yts >= non_digit_class] = non_digit_class

# Check if relabeling worked
print(f"Unique labels in ytr: {np.unique(ytr)}")
print(f"Count of labels == 10: {np.sum(ytr == 10)}")
print(f"Count of labels >= 10: {np.sum(ytr >= 10)}")

```

```

Unique labels in ytr: [ 0  1  2  3  4  5  6  7  8  9 10]
Count of labels == 10: 81600
Count of labels >= 10: 81600

```

Next, to reduce the training and test time, we will use only a small subset of the training and test data. Of course, you will not get great results with this small dataset. But, we can at least illustrate the basic concepts. Create new training and test data `Xtr1`, `ytr1`, `Xts1` and `yts1` with a randomly selected sample of digits and non-digits shown.

```

[89]: ntr_dig = 5000
      ntr_let = 1000
      nts_dig = 5000
      nts_let = 1000

      np.random.seed(42)
      train_indices = np.random.permutation(np.concatenate([
          np.random.choice(np.where(ytr < 10)[0], ntr_dig, replace=False),
          np.random.choice(np.where(ytr == 10)[0], ntr_let, replace=False)
      ]))
      test_indices = np.random.permutation(np.concatenate([
          np.random.choice(np.where(yts < 10)[0], nts_dig, replace=False),
          np.random.choice(np.where(yts == 10)[0], nts_let, replace=False)
      ]))

      Xtr1, ytr1 = Xtr[train_indices], ytr[train_indices]
      Xts1, yts1 = Xts[test_indices], yts[test_indices]

```

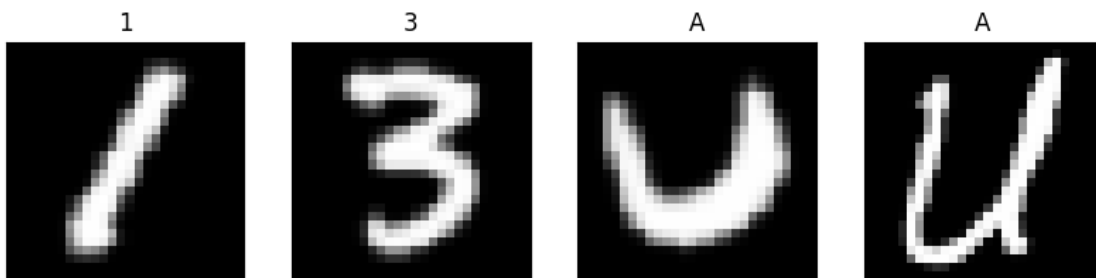
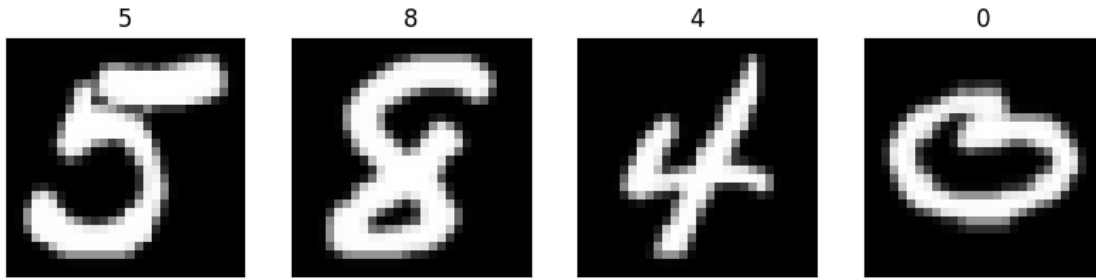
To confirm that selection worked, print

```

[90]: # TODO: Plot 8 random samples from the training data of the digits
      # Select random digits

      random_indices = np.random.randint(0, len(Xtr1), 8)
      plt.figure(figsize=(10, 20))
      for i in range(8):
          plt.subplot(2, 4, i + 1)
          plt_digit(Xtr1[random_indices[i]], classes[ytr1[random_indices[i]]])
      plt.show()

```



The training data above takes values from 0 to 255. Rescale the data from -1 to 1. This will get slightly better performance on the SVM. Save the scaled data into arrays `Xtr1` and `Xts1`.

```
[91]: # TODO: Rescale the data from -1 to 1
Xtr1 = Xtr1 / 255.0 * 2 - 1
Xts1 = Xts1 / 255.0 * 2 - 1

Xtr1 = Xtr1.reshape(Xtr1.shape[0], -1)
Xts1 = Xts1.reshape(Xts1.shape[0], -1)
```

1.4 Run the SVM classifier

First create the SVM classifier. Use an `rbf` classifier with `C=2.8` and `gamma=.0073`. We will look at how to select these parameters later.

```
[92]: from sklearn import svm

# TODO: Create a classifier: a support vector classifier
svc = svm.SVC(kernel='rbf', C=2.8, gamma=0.0073)
```

Fit the classifier using the scaled training data. SVMs are insanely slow to train. But, in this lab, we have kept the training size very small. So, the fitting should take about a minute or two.

```
[93]: # TODO: Fit the classifier on the training data.
svc.fit(Xtr1, ytr1)
```

```
[93]: SVC(C=2.8, gamma=0.0073)
```

Measure the accuracy on the test data. This too will take another huge amount of time. Print the accuracy. If you did everything right, you should get an accuracy of around 89%.

```
[94]: # TODO: Measure error on the test data
yts1_pred = svc.predict(Xts1)
acc = np.sum(yts1_pred == yts1) / len(yts1)
print(f"Accuracy: {acc:.2f}")
```

Accuracy: 0.93

The error rate is quite a bit higher than what we got in the digits only case. Actually, had we done a classifier using all 36 labels instead of collapsing the letters to a single class, the SVM classifier would have done much better. The reason is that the “letters” class is now extremely complex.

Print a confusion matrix. You should see that the error rate on the “letters” class is much higher.

```
[95]: from sklearn.metrics import confusion_matrix

# TODO: Print a confusion matrix
C = confusion_matrix(yts1, yts1_pred)

# TODO: Normalize the confusion matrix so that each row sums to one
C = C.astype(float) / C.sum(axis=1)[:, np.newaxis]
```



```
# TODO: Print the confusion matrix
print(C)
```

```
[[0.96320346 0.          0.          0.          0.          0.
  0.          0.          0.          0.          0.03679654]
 [0.          0.98198198 0.          0.          0.          0.
  0.          0.0018018  0.          0.          0.01621622]
 [0.          0.          0.91836735 0.          0.          0.
  0.00204082 0.          0.          0.          0.07959184]
 [0.          0.          0.00573614 0.94455067 0.          0.00573614
  0.          0.00573614 0.00191205 0.00191205 0.03441683]
 [0.          0.          0.          0.          0.96969697 0.
  0.          0.          0.          0.0040404  0.02626263]
 [0.00191571 0.          0.          0.01149425 0.          0.93678161
  0.          0.          0.          0.          0.04980843]
 [0.          0.          0.          0.          0.          0.
  0.95753715 0.          0.00212314 0.          0.0403397 ]
 [0.          0.00203666 0.00203666 0.          0.          0.
  0.          0.96334012 0.00407332 0.00610998 0.02240326]
 [0.          0.          0.00617284 0.00205761 0.          0.00205761
  0.          0.00205761 0.91358025 0.00411523 0.06995885]
 [0.0039604  0.          0.          0.00594059 0.          0.
  0.          0.01782178 0.0019802  0.93267327 0.03762376]
 [0.02        0.009       0.014       0.009       0.027       0.027
  0.023       0.003       0.014       0.029       0.825       ]]
```

Print: * What fraction of digits are mislabeled as non-digits?

* What fraction of letters are mislabeled as non-digits?

```
[98]: # TODO: Print above two error rates
print("Digits mislabeled as non-digits: ", np.sum(yts1_pred[yts1 < 10] == 10) /
      ↪ np.sum(yts1 < 10))
print("Letters mislabeled as non-digits: ", np.sum(yts1_pred[yts1 == 10] != 10) /
      ↪ np.sum(yts1 == 10))
```

Digits mislabeled as non-digits: 0.041

Letters mislabeled as non-digits: 0.175

1.5 Selecting gamma and C via Cross-Validation (Using For-Loops)

In the above example, and in the demo, we used a given `gamma` and `C` value. The selection of the parameters depend on the problem and decent performance of the SVM requires that you select these parameters carefully. The best way to select the parameters is via cross validation. Specifically, generally, one tries different values of `gamma` and `C` and selects the pair of values the lowest test error rate.

In the code below, we will try to use 3 values for `C` and `gamma` as specified in the arrays `C_test` and `gam_test`. For each `C` and `gamma` in these arrays, fit a model on the training data and measure the accuracy on the test data. Then, print the `C` and `gamma` that result in the best accuracy.

Normally, you would try a large number of values for each of the parameters, but an SVM is very slow to train – even with this small data set. So, we will just do 3 values of each. Even then, this could take 30 minutes or so to complete.

In this lab, you may do the parameter search over `C` and `gamma` in one of two ways: * This section: Use for loops and manually search over the parameters. This is more direct and you will see and control exactly what is happening. * Next section: Use the `GridSearchCV` method in the `sklearn` package. This takes a little reading, but once you learn this method, you can more easily use this for complex parameter searches.

You only need to submit the solutions to one of the two sections. Pick whichever one you want.

```
[99]: C_test = [0.1,1,10]
      gam_test = [0.001,0.01,0.1]

      nC = len(C_test)
      ngam = len(gam_test)
      acc = np.zeros((nC,ngam))

      # TODO: Measure and print the accuracy for each C and gamma value.
      # Store the results in acc
      #
      for i, C in enumerate(C_test):
          print(f"C: {C}")
          for j, gam in enumerate(gam_test):
              print(f"gam: {gam}")
              svc = svm.SVC(kernel='rbf', C=C, gamma=gam)
              svc.fit(Xtr1, ytr1)
              yts1_pred = svc.predict(Xts1)
              acc[i,j] = np.sum(yts1_pred == yts1) / len(yts1)
```

```
C: 0.1
gam: 0.001
gam: 0.01
gam: 0.1
C: 1
gam: 0.001
gam: 0.01
gam: 0.1
C: 10
gam: 0.001
gam: 0.01
gam: 0.1
```

```
[100]: # TODO: Print the accuracy matrix
      print(acc)
```

```
[[0.78933333 0.60166667 0.16666667]
 [0.8735      0.91216667 0.38966667]]
```

```
[0.9135      0.91733333 0.39166667]]
```

```
[102]: # TODO: Print the maximum accuracy and the corresponding best C and gamma
best_idx = np.unravel_index(np.argmax(acc), acc.shape)
best_C = C_test[best_idx[0]]
best_gamma = gam_test[best_idx[1]]
print(f"Best accuracy: {acc[best_idx]}")
print(f"Best C: {best_C}")
print(f"Best gamma: {best_gamma}")
```

```
Best accuracy: 0.9173333333333333
```

```
Best C: 10
```

```
Best gamma: 0.01
```

1.6 Using GridSearchCV (Optional Section)

In the previous section, you would have likely used `for`-loops to search over the different `C` and `gamma` values. Since this type of parameter search is so commonly used, `sklearn` has an excellent method `GridSearchCV` that can perform all the operations for you. In this lab, `GridSearchCV` is not that useful. But, once you get to more complex parameter searches, the `GridSearchCV` method can save you writing a lot of code. Importantly, `GridSearchCV` supports parallelization so that fits with different parameters can be fit at the same time. In this optional section, we will show how to use this method.

You do not have to do this section, if you did the previous section.

The `GridSearchCV` method does the train-test split in addition to the parameter search. In this case, you have already a fixed train-test split. So, you first need to combine the train and test data back into a single dataset.

Create arrays `X` and `y` from `Xtr1`, `Xts1`, `ytr` and `yts`. Use `np.vstack` and `np.hstack`.

```
[ ]: # TODO: Create combined trained and test data X and y.
# X = ...
# y = ...
```

Normally, `GridSearchCV` will do K -fold validation and automatically split the data into training and test in each fold. But, in this case, we want it to perform only one fold with a specific train-test split. To do this, we need to do the following: * Create a vector `test_fold` where `test_fold[i] = -1` for the samples `i` in the training data (this indicates that they should not be used as test data in any fold) and `test_fold[i] = 0` for the samples `i` in the test data (this indicates that they should be as test data in fold 0). * Call the method `ps = sklearn.model_selection.PredefinedSplit(test_fold)` to create a predefined test split object.

```
[17]: # TODO: Create a pre-defined test split object
# import sklearn.model_selection
# test_fold = ...
# ps = sklearn.model_selection.PredefinedSplit(test_fold)
```

Next, read about the `GridSearchCV` method to set up a classifier that includes searching over the parameter grid.

* For the `param_grid` parameter, you will want to create a dictionary to search over `C` and `gamma`.

You will also need to select the `kernel` parameter. * Set `cv = ps` to use the fixed train-test split.

* Set `verbose=10` to monitor the progress

```
[18]: # TODO: Create a GridSearchCV classifier
      # clf = ...
```

Fit the classifier using the `fit` method. The `fit` method will now search over all the parameters. This will take about 30 minutes.

```
[19]: # TODO: Fit the classifier
```

Print the `best_score_` and `best_params_` attributes of the classifier to find the best score and parameters

```
[20]: # TODO: Print the best parameter and score of the classifier
```

Finally, you can print the test and train score from the `cv_results_['mean_test_score']` and `cv_results_['mean_train_score']`.

```
[21]: # TODO: Print the mean test score for each parameter value.
```

```
[ ]:
```